IMPERIAL

Department of Chemical Engineering

Machine Learning for Chemical Engineering

Lecture 1b – Supervised Learning: Linear Regression

Data pre-processing (standardization)

Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$X = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad X \in \mathbb{R}^{151\times 2}$$

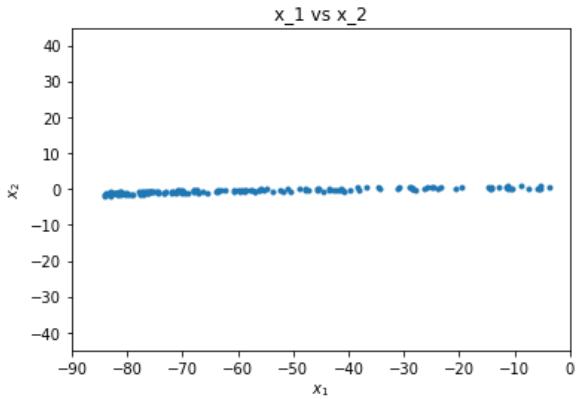
First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

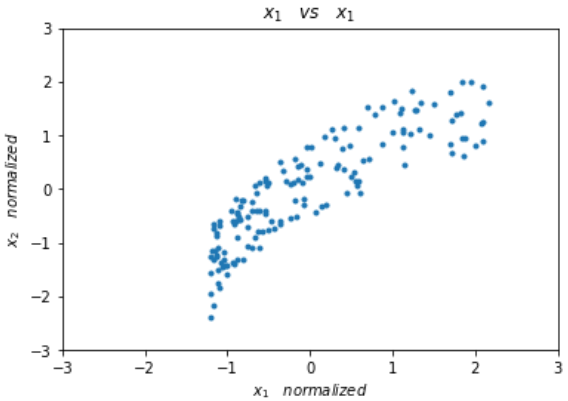
With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

Without standardization



With standardization



Data pre-processing (standardization)

We have an example with 151 points in 2-dimensions $\mathbf{x} \in \mathbb{R}^2$

$$X = \begin{bmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(151)} & x_2^{(151)} \end{bmatrix}, \qquad X \in \mathbb{R}^{151\times 2}$$

First step for any ML analysis (good practice) is **standardizing** the data:

$$\bar{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

With

$$\mu_j = \frac{1}{151}\sum_{i=1}^{151} x_j^{(i)}, \qquad \sigma_j = \sqrt{\frac{\sum_{i=1}^{151}\left(x_j^{(i)} - \mu_j\right)^2}{150}}$$

$$\bar{X} = \begin{bmatrix} \left(\bar{\mathbf{x}}^{(1)}\right)^T \\ \vdots \\ \left(\bar{\mathbf{x}}^{(151)}\right)^T \end{bmatrix} = \begin{bmatrix} \bar{x}_1^{(1)} & \bar{x}_2^{(1)} \\ \vdots & \vdots \\ \bar{x}_1^{(151)} & \bar{x}_2^{(151)} \end{bmatrix}$$

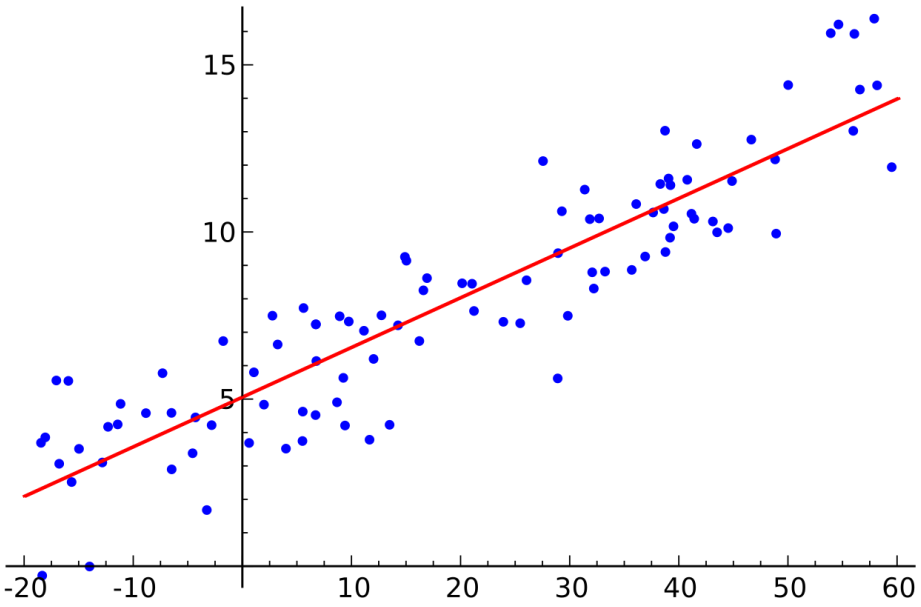
We assume all our data is standardized from now on $\mathbf{x} \to \bar{\mathbf{x}}$

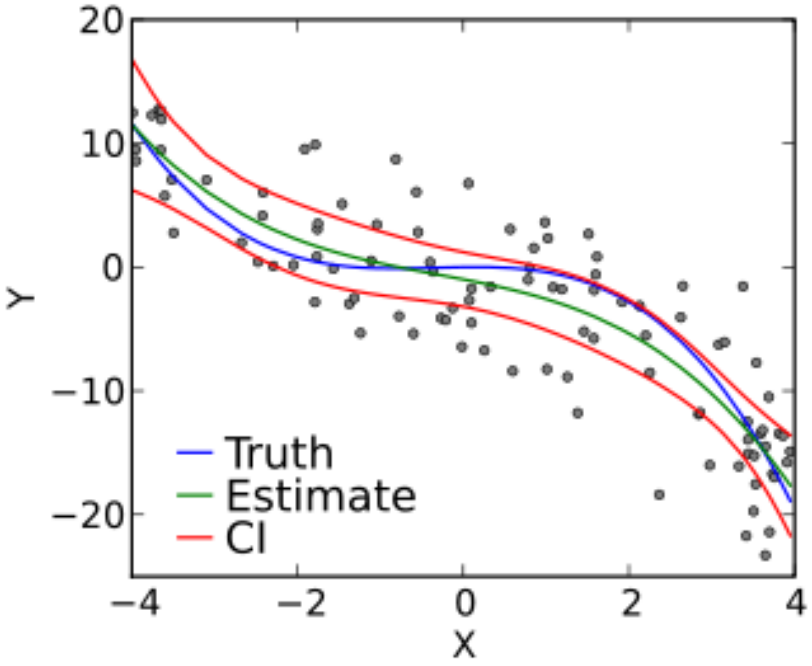
What is Machine Learning? – Supervised Learning

Supervised Learning

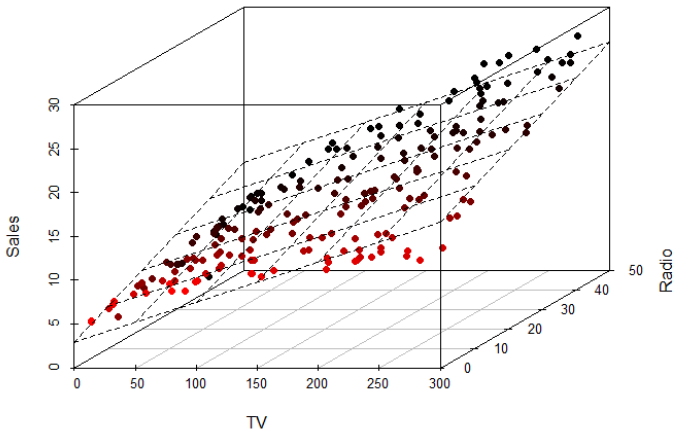
- Feeding the Learning agent with examples and trying to make it “generalize”.
- This can be thought as “**regression**” where the “machine” adjusts some parameters (or wights) to best capture the data.

Examples: Regression

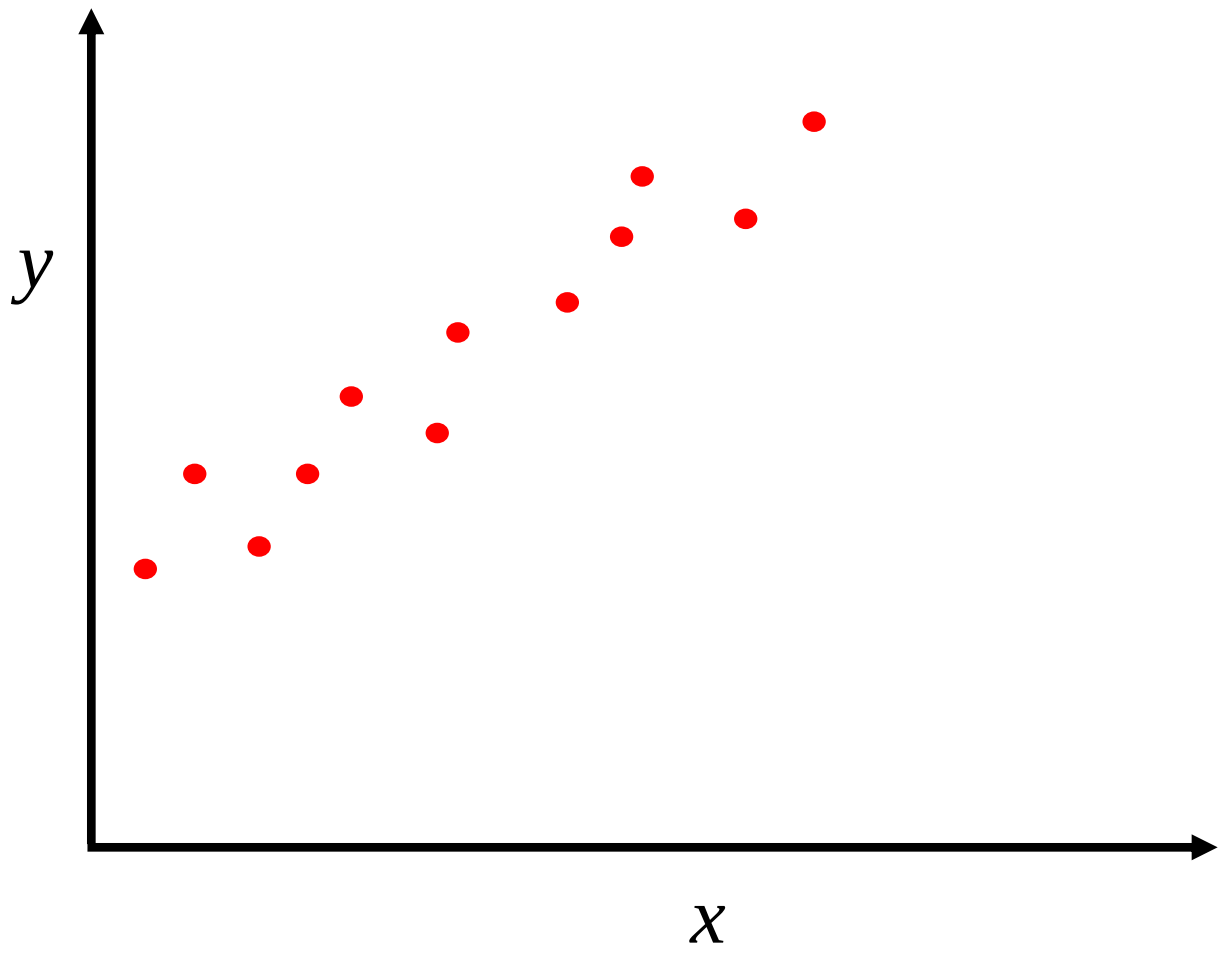




Linear Regression

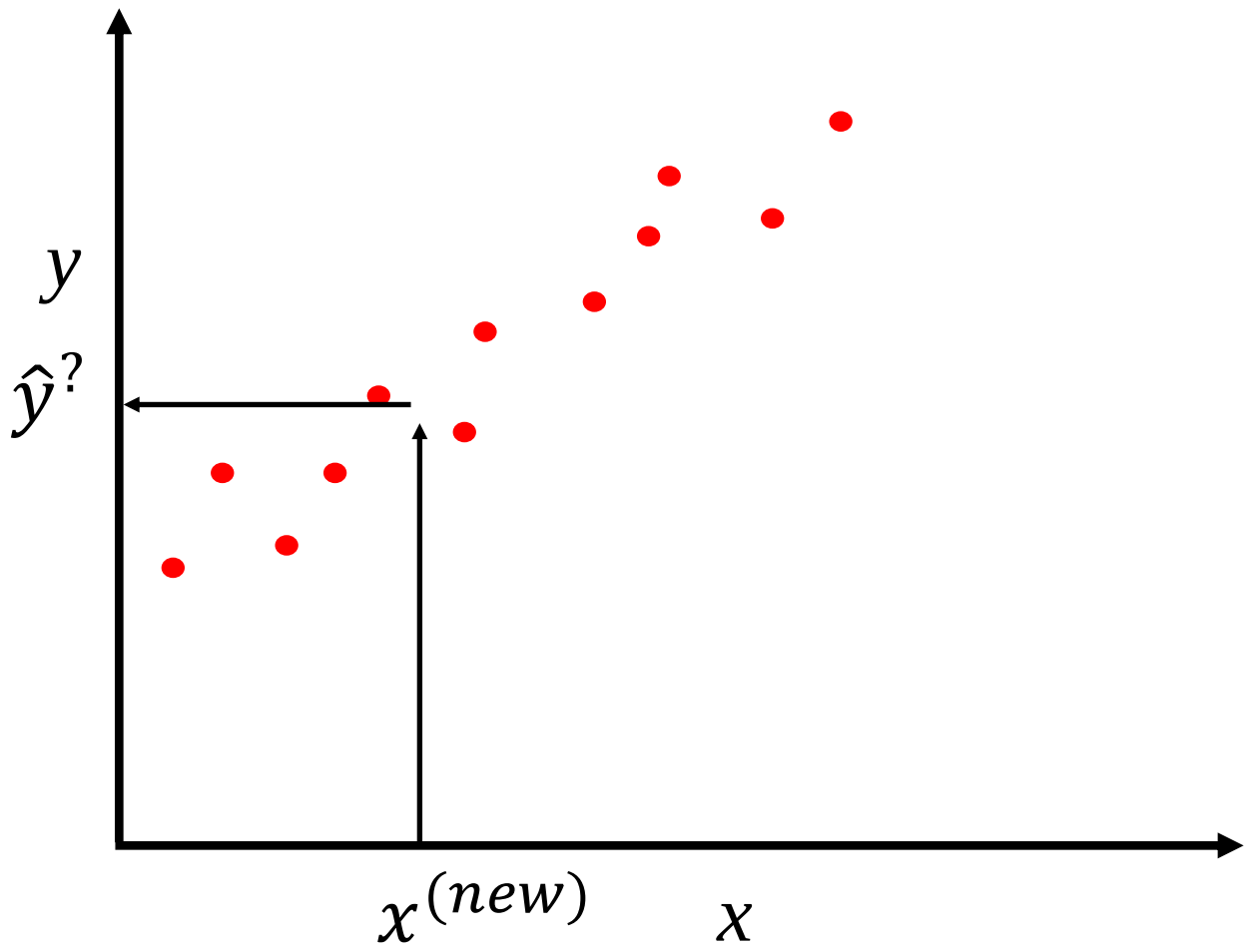


Supervised Learning – Linear Regression 1D example



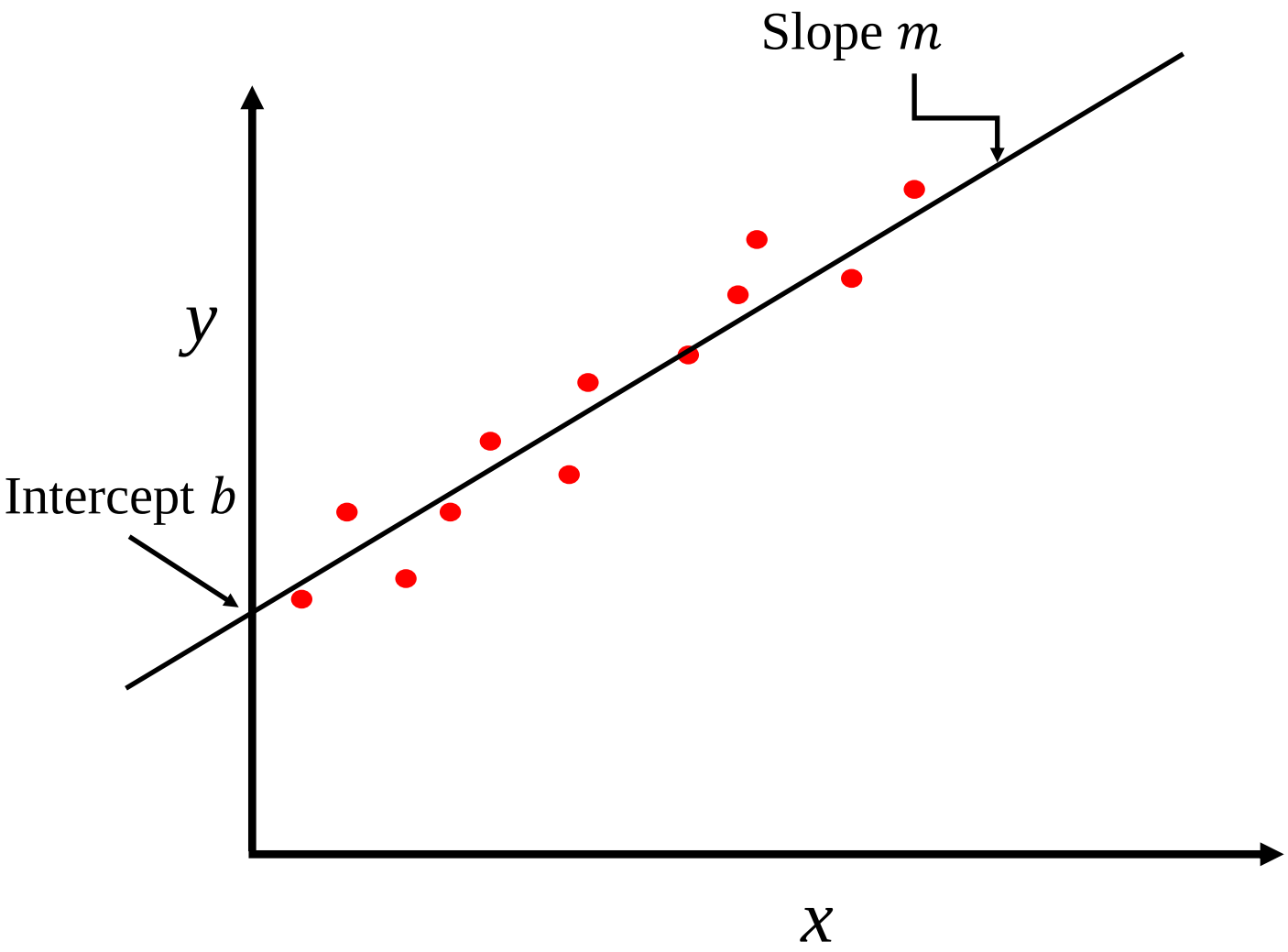
- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



- For several values of x we have their corresponding y value.

Supervised Learning – Linear Regression 1D example



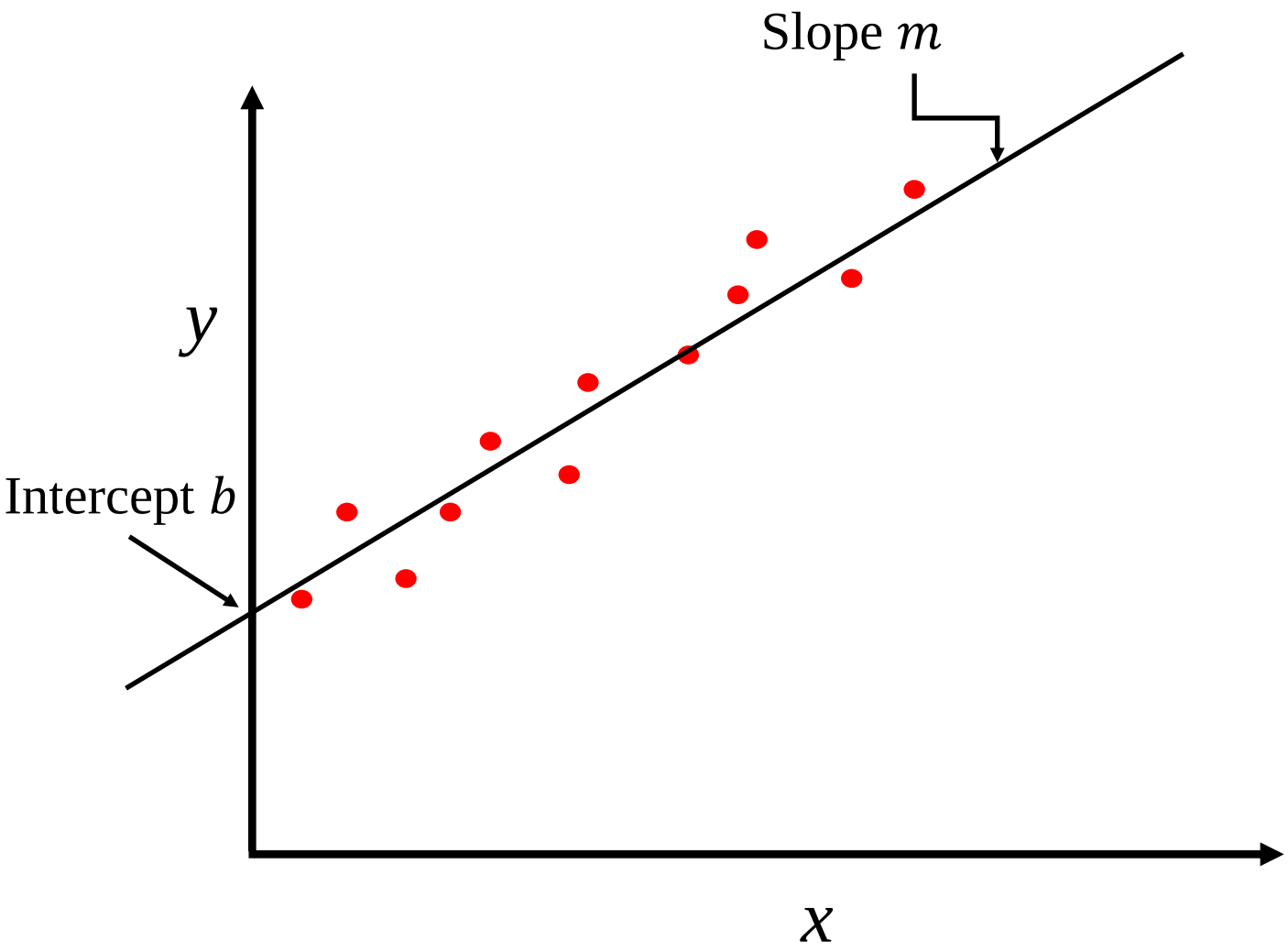
- For several values of x we have their corresponding y value.
- We build a model to predict y for new values of x.

$$\hat{y} = m\,x + b$$

- How do we find the best line (i.e. b and m) that models the data?

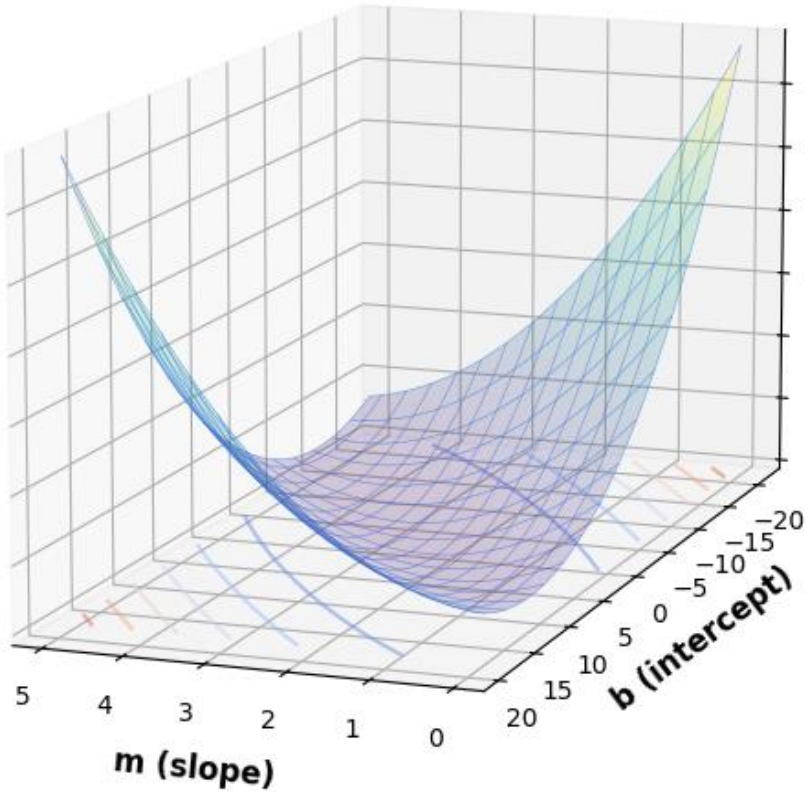
Optimization!

Supervised Learning – Linear Regression 1D example

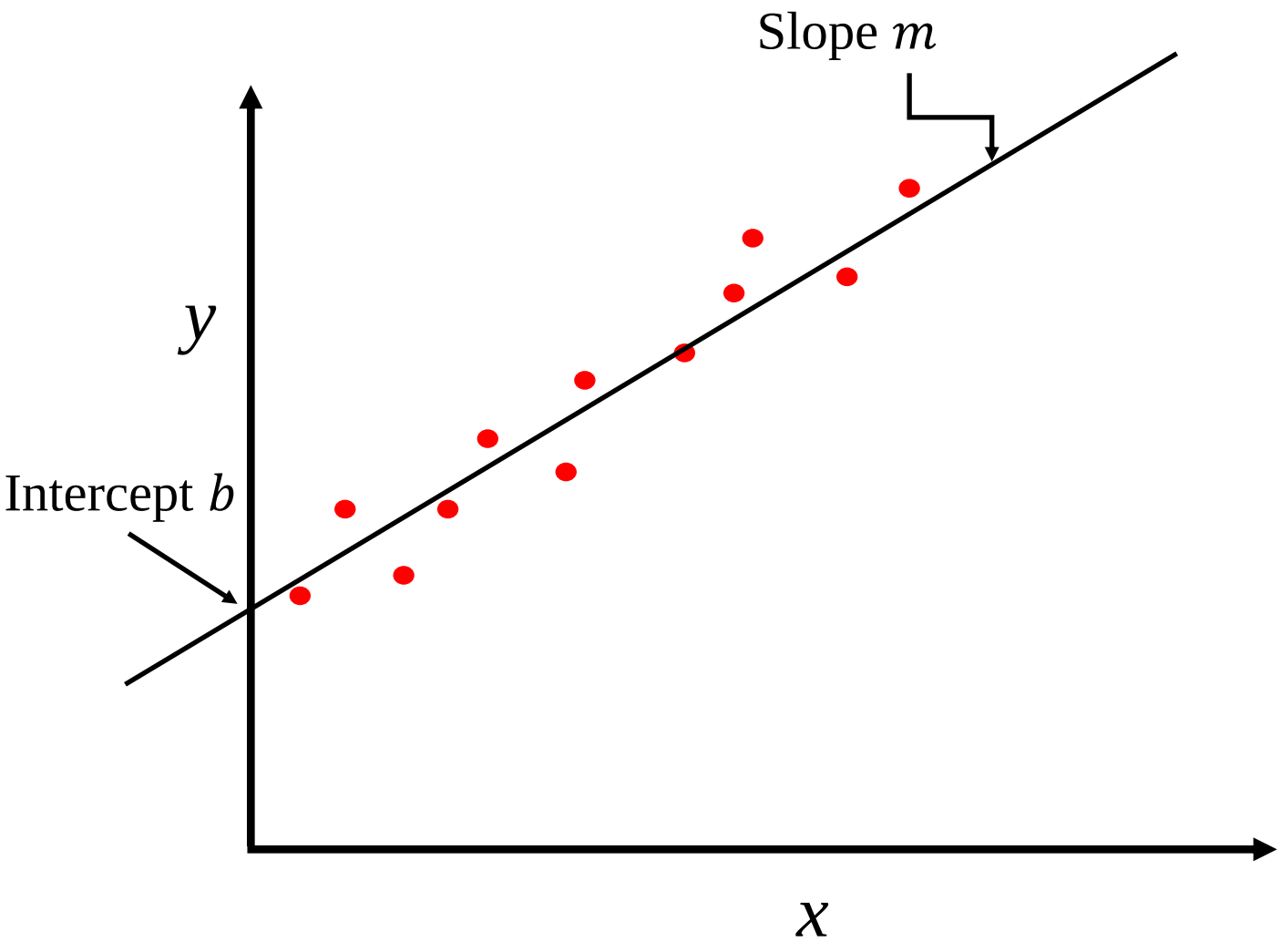


- We wish to find the best values for b and m.

$$\min_{m,b} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$



Supervised Learning – Linear Regression 1D example



- We wish to find the best values for b and m.

$$\min_{\boxed{m,b}} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Determine optimal values

$$f_{\text{obj}} := \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Solve:

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 0, \qquad \frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 0$$

Supervised Learning – Linear Regression 1D example

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 2\sum_{i=1}^{12} x^{(i)}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 2\sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right) = 0$$

We can rewrite as:

$$\frac{\partial f_{\text{obj}}}{\partial m} = m\sum_{i=1}^{12} \left(x^{(i)}\right)^2 + b\sum_{i=1}^{12} x^{(i)} - \sum_{i=1}^{12} x^{(i)}y^{(i)} = 0$$

$$\frac{\partial f_{\text{obj}}}{\partial b} = m\sum_{i=1}^{12} x^{(i)} + 12b - \sum_{i=1}^{12} y^{(i)} = 0$$

- We wish to find the best values for b and m.

$$\min_{\boxed{m,b}} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Determine optimal values

$$f_{\text{obj}} := \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

Solve:

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 0, \qquad \frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 0$$

Supervised Learning – Linear Regression 1D example

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial m} = 2\sum_{i=1}^{12} x^{(i)}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

$$\frac{\partial f_{\text{obj}}(m,b)}{\partial b} = 2\sum_{i=1}^{12}\left(mx^{(i)} + b - y^{(i)}\right) = 0$$

We can rewrite as:

$$\frac{\partial f_{\text{obj}}}{\partial m} = m\sum_{i=1}^{12}\left(x^{(i)}\right)^2 + b\sum_{i=1}^{12} x^{(i)} - \sum_{i=1}^{12} x^{(i)}y^{(i)} = 0$$

$$\frac{\partial f_{\text{obj}}}{\partial b} = m\sum_{i=1}^{12} x^{(i)} + 12b - \sum_{i=1}^{12} y^{(i)} = 0$$

- We wish to find the best values for b and m.

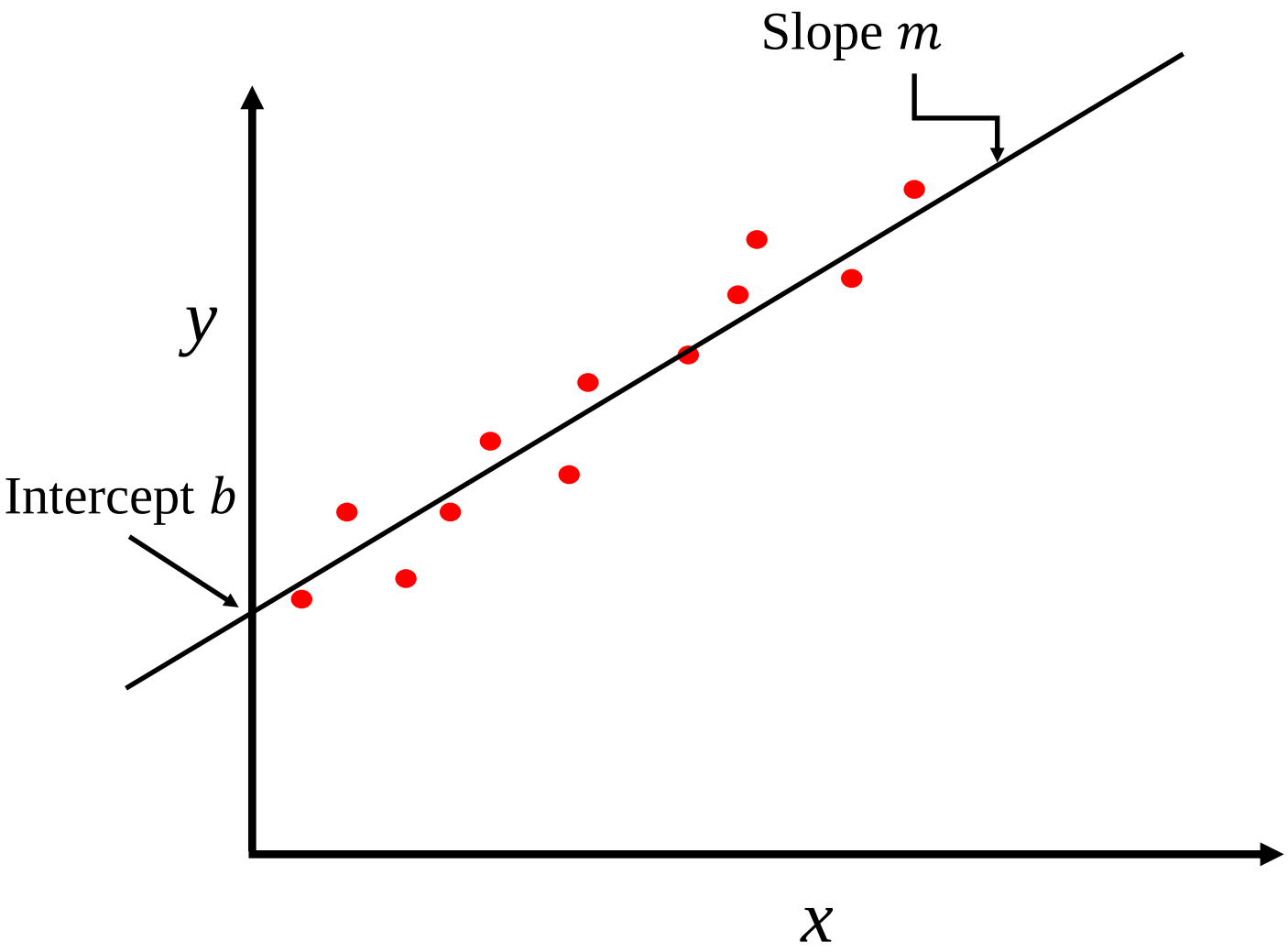
$$\min_{m,b}\sum_{i=1}^{12}\left(mx^{(i)} + b - y^{(i)}\right)^2$$

- Closed form solutions:

$$m = \frac{\sum_{i=1}^{12} x^{(i)}y^{(i)} - \frac{1}{12}\sum_{i=1}^{12} x^{(i)}\sum_{i=1}^{12} y^{(i)}}{\sum_{i=1}^{12}(x^{(i)})^2 - \frac{1}{12}\left(\sum_{i=1}^{12} x^{(i)}\right)^2}$$

$$b = \frac{\sum_{i=1}^{12} y^{(i)} - m\sum_{i=1}^{12} x^{(i)}}{12}$$

Supervised Learning – Linear Regression 1D example



- We wish to find the best values for b and m.

$$\min_{m,b} \sum_{i=1}^{12} \left(mx^{(i)} + b - y^{(i)}\right)^2$$

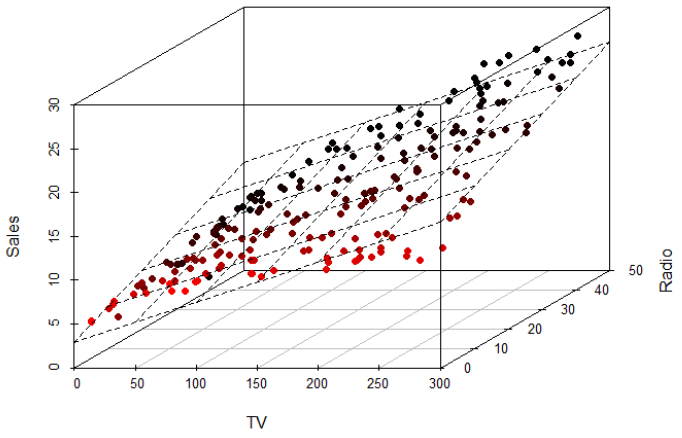
- Closed form solutions:

$$m = \frac{\sum_{i=1}^{12}\left(x^{(i)} - \overline{x}\right)\left(y^{(i)} - \overline{y}\right)}{\sum_{i=1}^{12}\left(x^{(i)} - \overline{x}\right)^2}$$

$$b = \overline{y} - m\overline{x}$$

Where $\overline{x}$ and $\overline{y}$ are the average of the $x^{(i)}$ and $y^{(i)}$, respectively.

Linear regression with multiple inputs

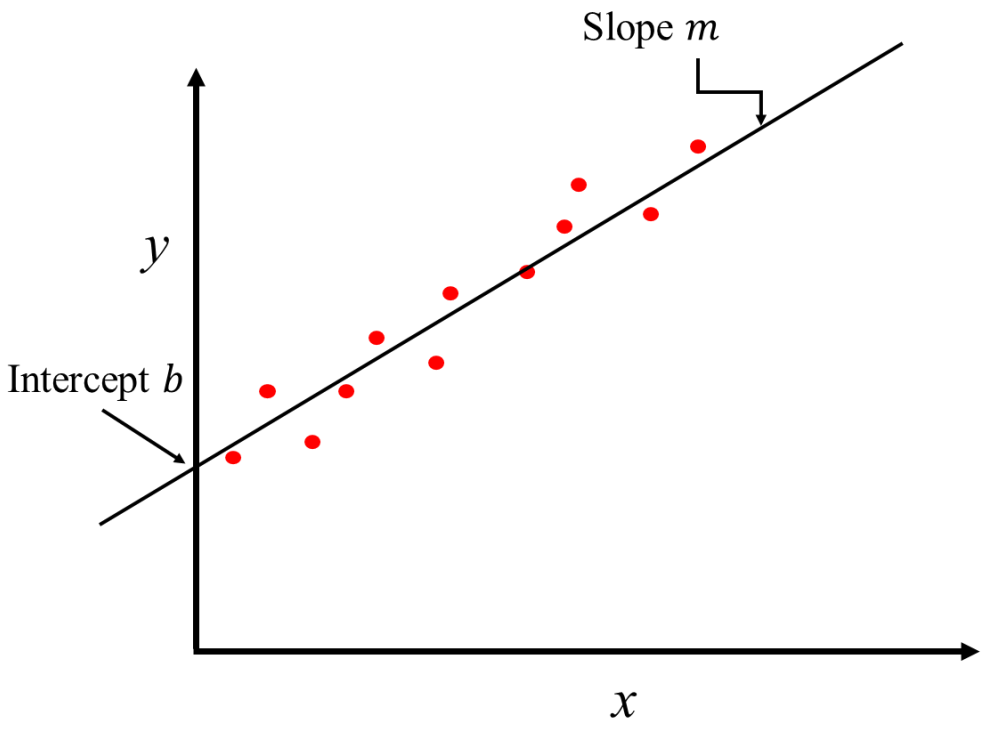


Supervised Learning – Linear Regression

We went through a 1D example where the input data $x^{(i)} \in \mathbb{R}$ was a scalar

$$\text{Data: } \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n_d)}, y^{(n_d)})\}$$

n_d refers to the number of datapoints.



Supervised Learning – Linear Regression

We went through a 1D example where the input data $x^{(i)} \in \mathbb{R}$ was a scalar

$$\text{Data: } \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n_d)}, y^{(n_d)})\}$$

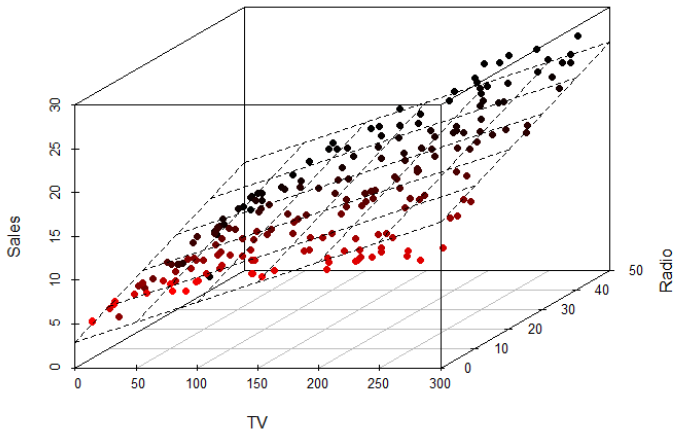
n_d refers to the number of datapoints.

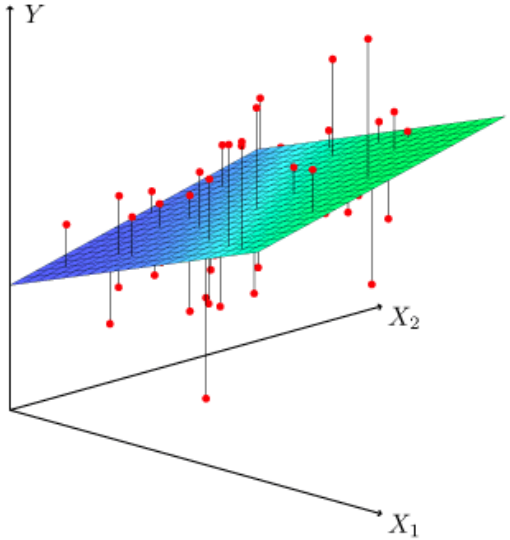
What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

n_x is the dimension of our input.





2D visualization: $n_x = 2$

Supervised Learning – Linear Regression

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Now we must determine:

$$\hat{y} \approx m_1 x_1 + m_2 x_2 + \cdots + m_{n_x} x_{n_x} + b$$

Parameters to be tuned

Supervised Learning – Linear Regression

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Now we must determine:

Bias term

$$\hat{y} \approx m_1 x_1 + m_2 x_2 + \cdots + m_{n_x} x_{n_x} + b$$

One "slope"
per dimension

Supervised Learning – Linear Regression

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Change of notation:

$$\hat{y} \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0, \qquad \boldsymbol{\theta} = \left[\theta_0, \theta_2, \dots, \theta_{n_x}\right]^T$$

Now I wish to find the best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \Big(\underbrace{\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0}_{\text{Model}} - \overbrace{y^{(i)}}^{\text{Data}} \Big)^2$$

Supervised Learning – Linear Regression

What happens when each of our **input points is a vector** $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$?

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Change of notation:

$$\hat{y} \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0, \qquad \boldsymbol{\theta} = \left[\theta_0, \theta_2, \dots, \theta_{n_x}\right]^T$$

Now I wish to find the best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

$$\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \qquad i = 0, \dots n_x$$

<u>Also analytical solution!</u>

Supervised Learning – Linear Regression

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{pmatrix}$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Supervised Learning – Linear Regression

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{pmatrix}$$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ **Analytical solution!**

Supervised Learning – Linear Regression

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation: $y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \ i = 1, \dots, n_d$

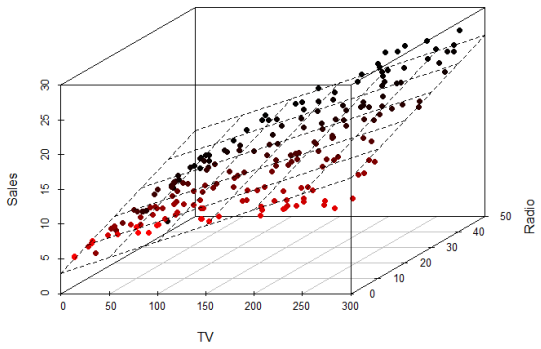
$$\boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{pmatrix}$$

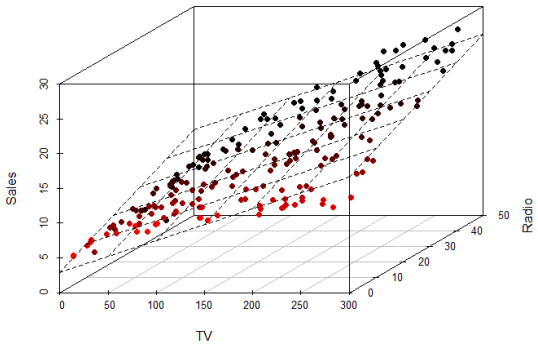
Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

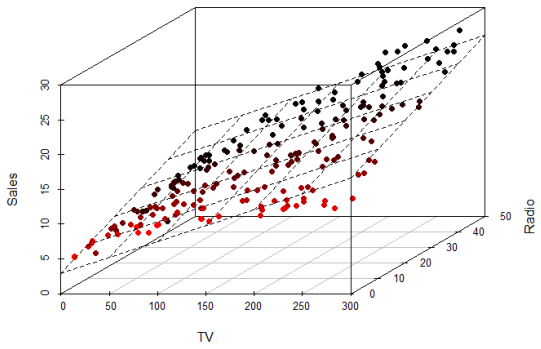
New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

If n_x is very large $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} \approx 0$ is solved by an iterative method.

Linear regression with multiple inputs & outputs







Supervised Learning – Linear Regression

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}, y \in \mathbb{R}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n_d)}, y^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

Solution: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \ i = 0, \dots n_x$

Another bit of notation:

$$y^{(i)} \approx \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + (1)\theta_0, \qquad i = 1, \dots, n_d$$

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n_x} \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{pmatrix}$$

Solution: $\boldsymbol{\theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\theta}^*$

Supervised Learning – Linear Regression

For $\mathbf{x}^{(i)} \in \mathbb{R}^{n_x}$, $\mathbf{y} \in \mathbb{R}^{n_y}$

$$\text{Data: } \{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(n_d)}, \mathbf{y}^{(n_d)})\}$$

Where $\mathbf{x}^{(i)} = \left[x_1^{(i)}, x_2^{(i)}, \dots, x_{n_x}^{(i)}\right]^T$

Best values for $\boldsymbol{\theta}$:

$$\min_{\boldsymbol{\Theta}} \sum_{j=1}^{n_y} \sum_{i=1}^{n_d} \left(\theta_{1,j} x_1^{(i)} + \cdots + \theta_{n_x,j} x_{n_x}^{(i)} + \theta_{0,j} - y_j^{(i)}\right)^2$$

Solution $\frac{\partial f_{\text{obj}}(\boldsymbol{\Theta})}{\partial \theta_{i,j}} = 0,\ i = 0, \dots n_x,\ j = 1, \dots, n_y$

Another bit of notation:

$$y_j^{(i)} \approx \theta_{1,j} x_1^{(i)} + \theta_{2,j} x_2^{(i)} + \cdots + \theta_{n_x,j} x_{n_x}^{(i)} + (1)\theta_{0,j}, \qquad i = 1, \dots, n_d, \qquad j = 1, \dots, n_y$$

$$\boldsymbol{\Theta} = \begin{pmatrix} \boldsymbol{\theta}_0 \\ \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_{n_x} \end{pmatrix}, \qquad \mathbf{Y} = \begin{pmatrix} \mathbf{y}^{(1)} \\ \mathbf{y}^{(2)} \\ \vdots \\ \mathbf{y}^{(n_d)} \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} \left(\mathbf{x}^{(1)}\right)^T \\ \left(\mathbf{x}^{(2)}\right)^T \\ \vdots \\ \left(\mathbf{x}^{(n_d)}\right)^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_{n_x}^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_{n_x}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n_d)} & \cdots & x_{n_x}^{(n_d)} \end{pmatrix}$$

Solution: $\boldsymbol{\Theta}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$ New observation: $\mathbf{y}_{\text{estimated}} = \mathbf{x}_{\text{new}}^T \boldsymbol{\Theta}^*$

Code example

```
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, Y):

        # GP variable definitions
        self.X, self.Y              = X, Y
        self.n_point, self.nx_dim   = X.shape[0], X.shape[1]
        # add bias term
        X = np.hstack(( np.ones((X.shape[0],1)), X ))

        # Training model (finding parameter)
        self.beta_linear = np.linalg.inv(X.T@X + np.eye(X.shape[1])*1e-6)@X.T@Y
```

Supervised Learning – Linear Regression

$$\mathbf{X} = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n_d)} \end{pmatrix},$$

```
1 Xtrain_1D
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

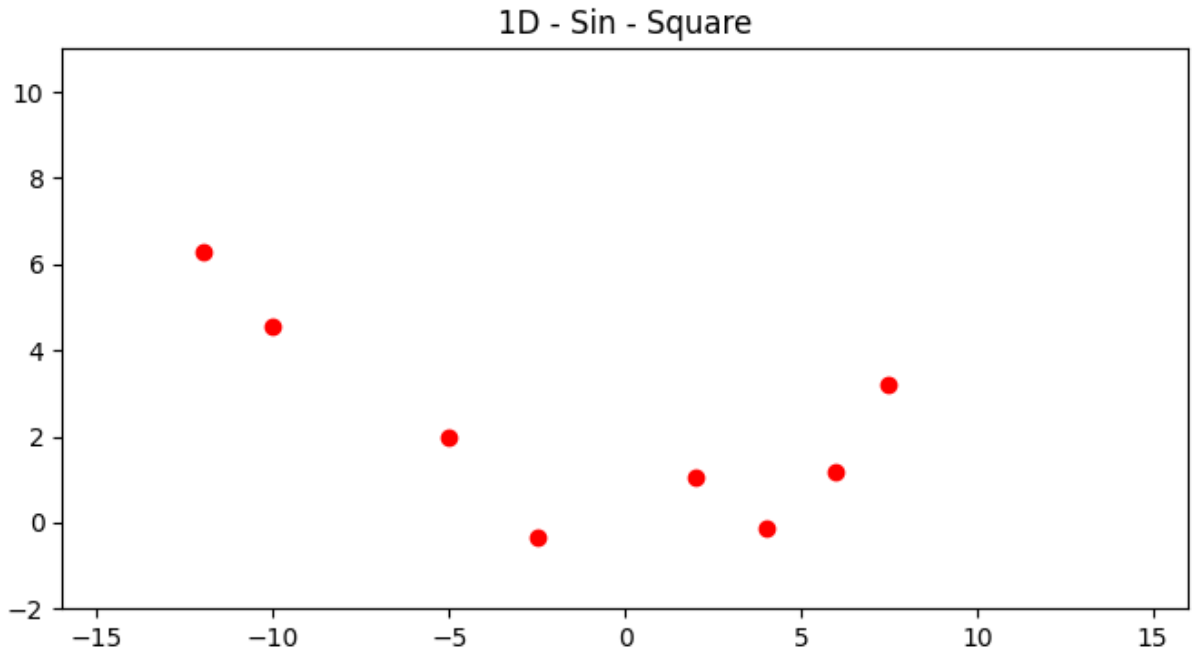
$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}$$

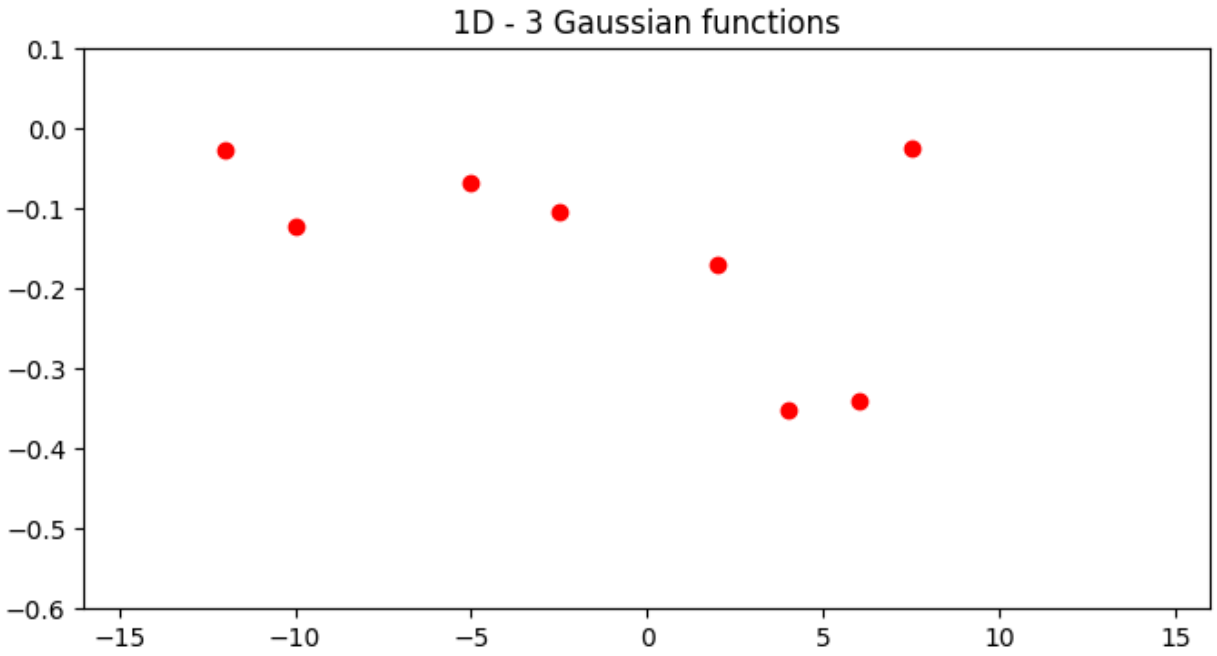
```
1 Ytrain_SS
```

```
array([[ 6.29657292],
       [ 4.54402111],
       [ 1.95892427],
       [-0.34847214],
       [ 1.06929743],
       [-0.1168025 ],
       [ 1.1605845 ],
       [ 3.18799998]])
```

```
1 Ytrain_3Gauss
```

```
array([[-0.02706706],
       [-0.12130613],
       [-0.06647686],
       [-0.10429972],
       [-0.1706667 ],
       [-0.35108337],
       [-0.33979135],
       [-0.02460617]])
```





Supervised Learning – Linear Regression

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, Y):

        # GP variable definitions
        self.X, self.Y                  = X, Y
        self.n_point, self.nx_dim       = X.shape[0], X.shape[1]
        # add bias term
        X = np.hstack(( np.ones((X.shape[0],1)), X ))

        # Training model (finding parameter)
        self.beta_linear = np.linalg.inv(X.T@X + np.eye(X.shape[1])*1e-6)@X.T@Y

    #####################
    # --- Prediction --- #
    #####################

    def predict(self, xnew):
        '''
        redicts y based on xnew
        '''
        # adding bias
        xnew = np.hstack((1, xnew))

        return self.beta_linear.T@xnew
```

Reshaping and extracting data information

$$\mathbf{X} = \begin{pmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{pmatrix}$$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T\boldsymbol{\theta}$

```python
Xtrain_1D
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

```python
Ytrain_SS
```

```
array([[ 6.29657292],
       [ 4.54402111],
       [ 1.95892427],
       [-0.34847214],
       [ 1.06929743],
       [-0.1168025 ],
       [ 1.1605845 ],
       [ 3.18799998]])
```

Supervised Learning – Linear Regression

```python
#####################
# Linear Regression #
#####################

class Linear_Regression_model:

    def __init__(self, X, Y):

        # GP variable definitions
        self.X, self.Y              = X, Y
        self.n_point, self.nx_dim   = X.shape[0], X.shape[1]
        # add bias term
        X = np.hstack(( np.ones((X.shape[0],1)), X ))

        # Training model (finding parameter)
        self.beta_linear = np.linalg.inv(X.T@X + np.eye(X.shape[1])*1e-6)@X.T@Y

    #####################
    # --- Prediction --- #
    #####################

    def predict(self, xnew):
        '''
        redicts y based on xnew
        '''
        # adding bias
        xnew = np.hstack((1, xnew))

        return self.beta_linear.T@xnew
```

```python
##########################
# Training linear models #
##########################

LR_model_SS1D = Linear_Regression_model(Xtrain_1D, Ytrain_SS)
LR_model_3G   = Linear_Regression_model(Xtrain_1D, Ytrain_3Gauss)
```

Reshaping and extracting data information

$$\mathbf{X} = \begin{pmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ & \vdots \\ 1 & x_1^{(n_d)} \end{pmatrix}$$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

New observation: $y_{\text{estimated}} = \mathbf{x}_{\text{new}}^T\boldsymbol{\theta}$

Supervised Learning – Linear Regression

$$\mathbf{X} = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n_d)} \end{pmatrix},$$

```
1 Xtrain_1D
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}$$

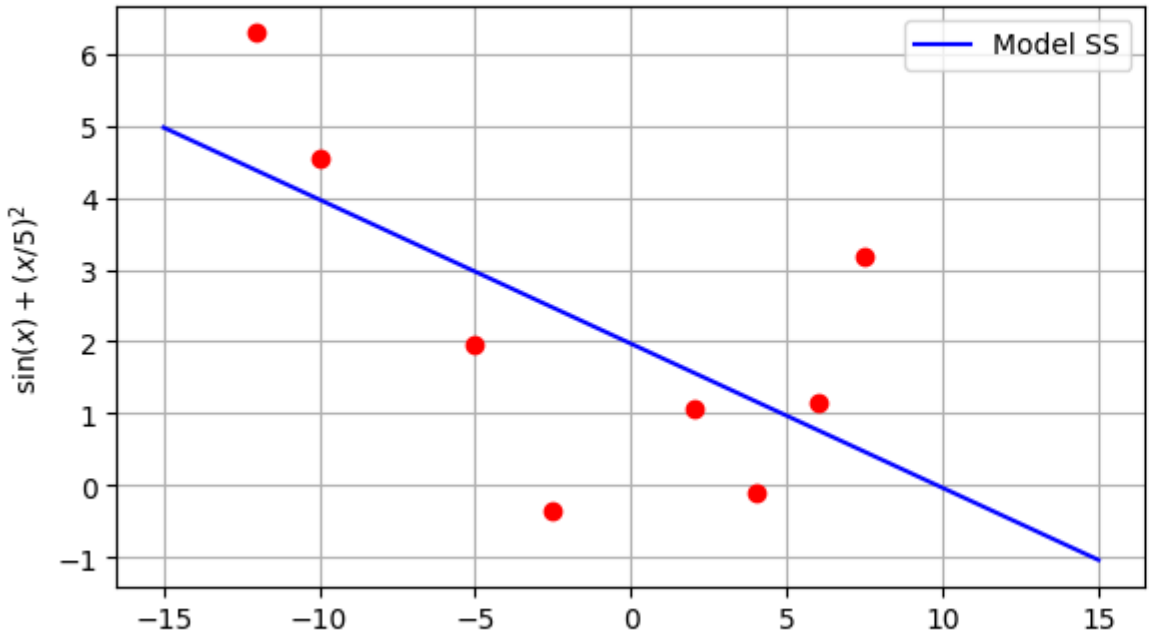
```
1 Ytrain_SS
```

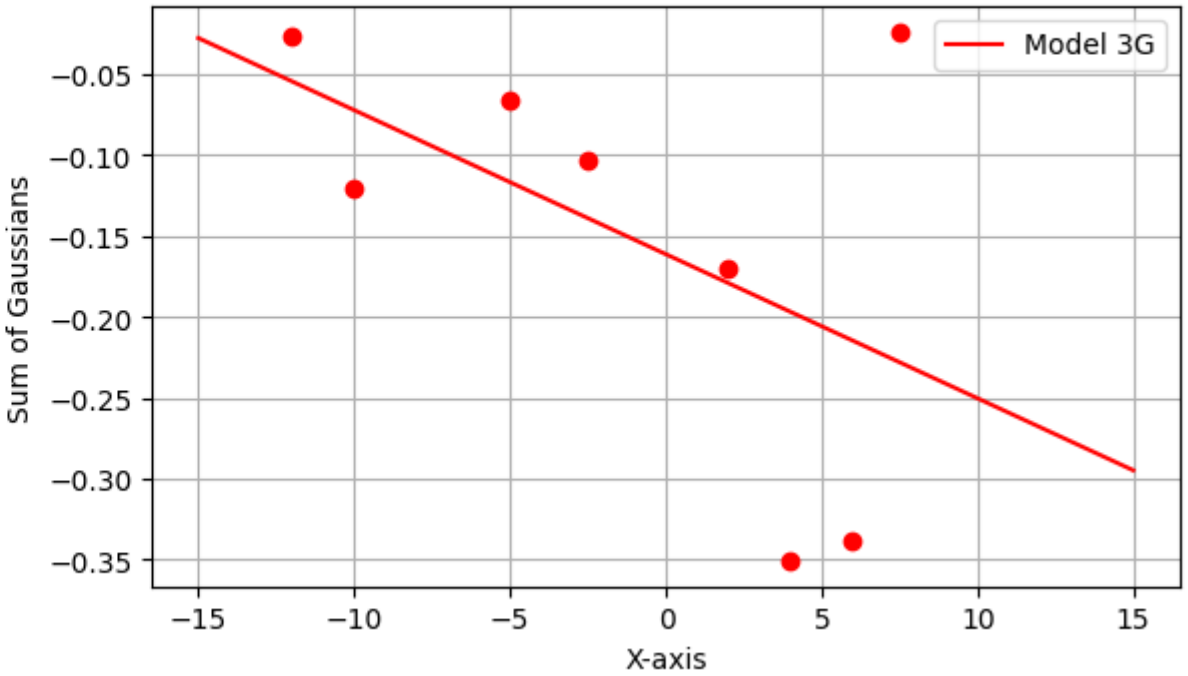
```
array([[ 6.29657292],
       [ 4.54402111],
       [ 1.95892427],
       [-0.34847214],
       [ 1.06929743],
       [-0.1168025 ],
       [ 1.1605845 ],
       [ 3.18799998]])
```

```
1 Ytrain_3Gauss
```

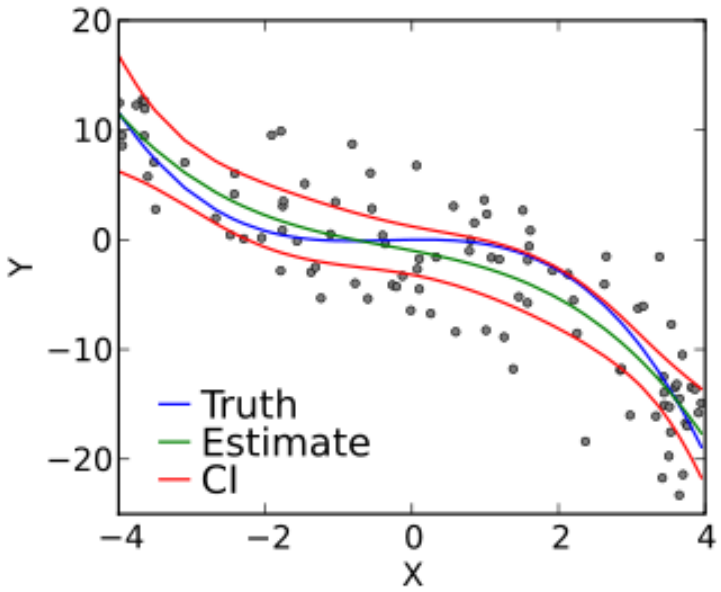
```
array([[-0.02706706],
       [-0.12130613],
       [-0.06647686],
       [-0.10429972],
       [-0.1706667 ],
       [-0.35108337],
       [-0.33979135],
       [-0.02460617]])
```

Linear Regression





Generalized linear regression



Supervised Learning – Linear Regression

- Can we capture nonlinear trends in our data with **linear regression**?

Of course we can!

For example, we currently have:

$$y \approx \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_{n_x} x_{n_x} + \theta_0$$

We could however do:

$$y \approx \theta_1 x_1 + \theta_{1,1} x_1^2 + \theta_{1,2} x_1 x_2 + \theta_2 x_2 + \theta_{2,2} x_2^2 + \cdots + \theta_{n_x,n_x} x_{n_x}^2 + \theta_0$$

Notice you can make many combinations or even functions:

$$y \approx \theta_1 \phi_1(\mathbf{x}) + \cdots + \theta_{n_\phi} \phi_{n_\phi}(\mathbf{x}) + \theta_0 b$$

Practically any function of your vector $\mathbf{x}$, e.g.:

$$\phi(\mathbf{x}) \coloneqq \sin(\mathbf{x}), \exp(\mathbf{x}), \log(\mathbf{x}), \ldots$$

Supervised Learning – Linear Regression

```python
class BasisF_Regression_model:

    def __init__(self, X, Y):

        # BF variable definitions
        self.X, self.Y              = X, Y
        self.n_point, self.nx_dim = X.shape[0], X.shape[1]
        m_X                         = np.mean(np.abs(X), axis=0)[0]
        self.m_X                    = m_X

        # add quadratic model
        XBF = np.hstack((X, X*X ))
        for dim_i in range(X.shape[1]-1):
            for dim_j in range(dim_i+1, X.shape[1]):
                XBF = np.hstack((XBF, X[:,dim_i].reshape(X.shape[0],1)*X[:,dim_j].reshape(X.shape[0],1)))
        # add cubic model
        for dim_i in range(X.shape[1]):
            for dim_j in range(dim_i, X.shape[1]):
                for dim_k in range(dim_j, X.shape[1]):
                    XBF = np.hstack((XBF, (X[:,dim_i].reshape(X.shape[0],1)*
                                           X[:,dim_j].reshape(X.shape[0],1)*
                                           X[:,dim_k].reshape(X.shape[0],1)
                                          )))
        # add sin term
        XBF = np.hstack(( XBF, np.sin(X) ))
        XBF = np.hstack(( XBF, np.cos(X) ))
        # add bias term
        XBF = np.hstack(( np.ones((XBF.shape[0],1)), XBF ))

        # Training model (finding parameter)
        self.beta_BF = np.linalg.inv(XBF.T@XBF + np.eye(XBF.shape[1])*1e-6)@XBF.T@Y
```

Reshaping and extracting data information

$$\mathbf{X} = \begin{pmatrix} \boldsymbol{\phi}\left(x^{(1)}\right)^T \\ \boldsymbol{\phi}\left(x^{(2)}\right)^T \\ \vdots \\ \boldsymbol{\phi}\left(x^{(n_d)}\right)^T \end{pmatrix}, \mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}$$

$$\boldsymbol{\phi}(\mathbf{x}) := (x, x^2)$$

$$\boldsymbol{\phi}(\mathbf{x}) := (x, x^2, x^3)$$

$$\boldsymbol{\phi}(\mathbf{x}) := \left(x, x^2, x^3, \sin(x), \cos(x)\right)$$

$$\boldsymbol{\phi}(\mathbf{x}) := \left(1, x, x^2, x^3, \sin(x), \cos(x)\right)$$

Solution: $\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Supervised Learning – Linear Regression

$$\mathbf{X} = \begin{pmatrix} \boldsymbol{\phi}\left(x^{(1)}\right)^T \\ \boldsymbol{\phi}\left(x^{(2)}\right)^T \\ \vdots \\ \boldsymbol{\phi}\left(x^{(n_d)}\right)^T \end{pmatrix},$$

```
1 Xtrain_1D
```

```
array([[-12. ],
       [-10. ],
       [ -5. ],
       [ -2.5],
       [  2. ],
       [  4. ],
       [  6. ],
       [  7.5]])
```

$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n_d)} \end{pmatrix}$$

```
1 Ytrain_SS
```

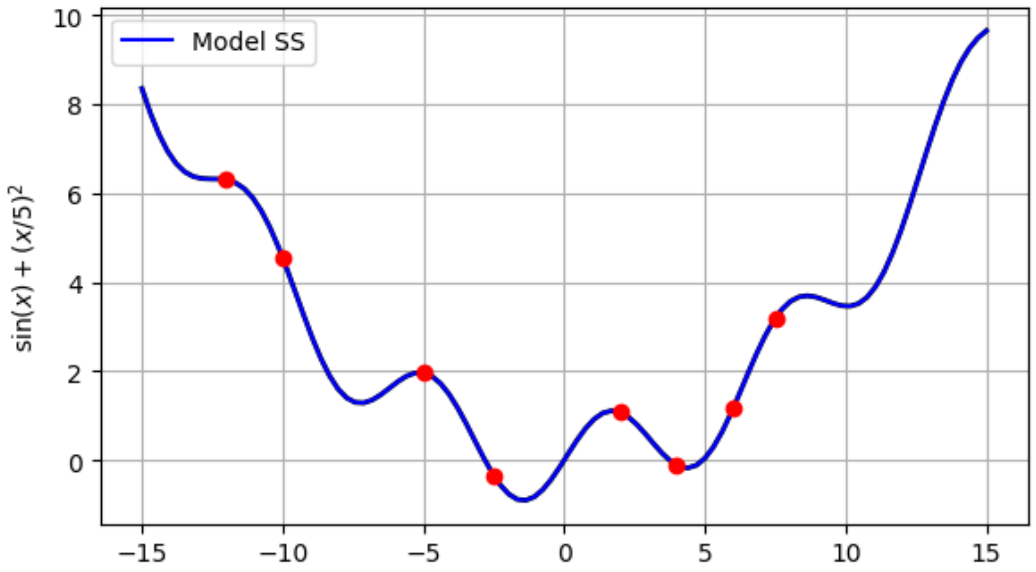
```
array([[ 6.29657292],
       [ 4.54402111],
       [ 1.95892427],
       [-0.34847214],
       [ 1.06929743],
       [-0.1168025 ],
       [ 1.1605845 ],
       [ 3.18799998]])
```

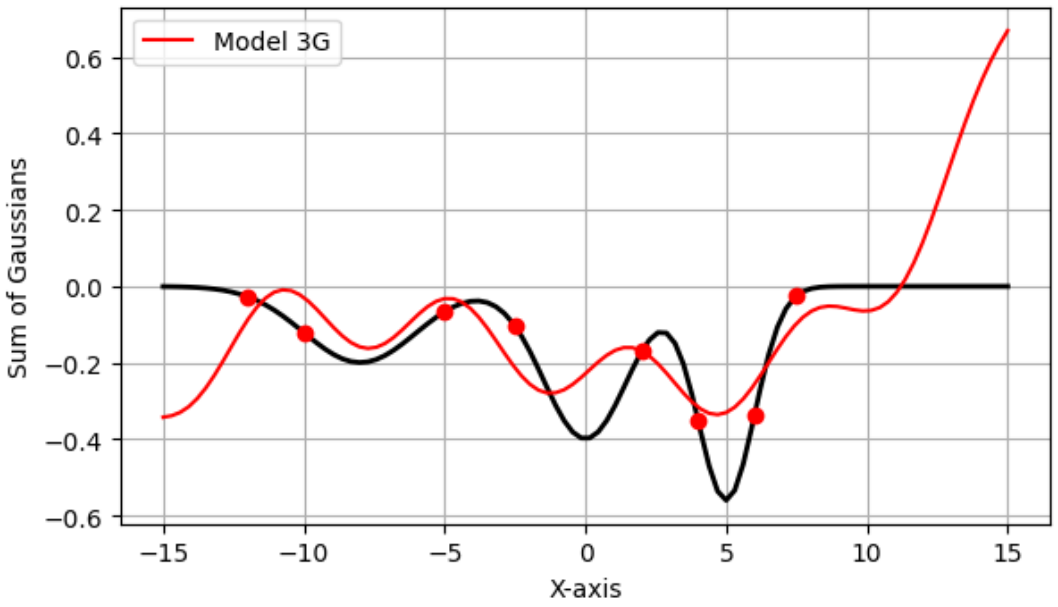
```
1 Ytrain_3Gauss
```

```
array([[-0.02706706],
       [-0.12130613],
       [-0.06647686],
       [-0.10429972],
       [-0.1706667 ],
       [-0.35108337],
       [-0.33979135],
       [-0.02460617]])
```

$$\boldsymbol{\phi}(\mathbf{x}) := \left(1, x, x^2, x^3, \sin(x), \cos(x)\right)^T$$

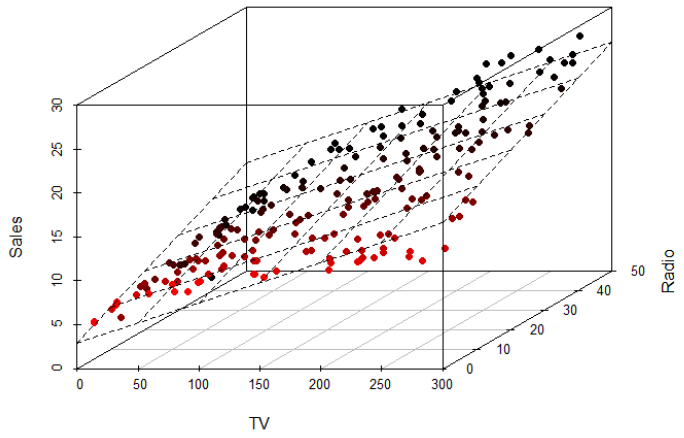
Basis Functions Regression





Summary – linear regression

Data: $\left\{\left(\mathbf{x}^{(1)}, y^{(1)}\right), \left(\mathbf{x}^{(2)}, y^{(2)}\right), \ldots, \left(\mathbf{x}^{(n_d)}, y^{(n_d)}\right)\right\}$



Least Squares

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \sum_{i=1}^{n_d} \left(\theta_1 x_1^{(i)} + \cdots + \theta_{n_x} x_{n_x}^{(i)} + \theta_0 - y^{(i)}\right)^2$$

$$\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0, \qquad i = 0, \ldots n_x$$

Matrix form:

$$\min_{\boldsymbol{\theta}} f_{\text{obj}} := \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

Fitted values

$$\hat{\mathbf{y}} = \theta_0 + \sum_{i=1}^{n_x} \theta_i x_i\,, \qquad \text{or,} \qquad \hat{\mathbf{y}} = \mathbf{x}^T \boldsymbol{\theta}$$

Note: $\frac{\partial f_{\text{obj}}(\boldsymbol{\theta})}{\partial \theta_i} = 0$ is solved by an iterative procedure if $\boldsymbol{\theta}$ or $\mathbf{X}$ is too large.

For nonlinear functions we can:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n_d} \left(\theta_1 \phi_1(\mathbf{x}) + \cdots + \theta_{n_\phi} \phi_{n_\phi}(\mathbf{x}) + \theta_0 b - y^{(i)}\right)^2$$

$$\hat{\mathbf{y}} = \theta_0 + \sum_{i=1}^{n_\phi} \theta_i \phi_i(\mathbf{x})$$